

Complete DSA Learning Order

Tiered Roadmap for Coding Interview Preparation (Examples in C#)

This is a complete, dependency-ordered topic list for learning Data Structures and Algorithms for coding interviews, organized into three tiers based on real interview frequency. Tier 1 topics appear constantly in interviews and deserve deep, repeated practice. Tier 2 topics appear occasionally and should be understood solidly but don't need the same repetition. Tier 3 topics rarely appear directly but are worth recognizing by name. All examples and practice should be done in C#, to reinforce your existing .NET track rather than splitting attention across languages.

How to Practice: Pattern-First, Not Problem-First

Unlike the .NET roadmap (one evolving project), DSA mastery comes from repetition across many small problems, not one big build. For every topic below, do the listed practice problems on a real platform (LeetCode unless noted), in order, without looking at solutions first. Spend at least 20-30 minutes stuck before checking a hint. The goal of each topic isn't "solve these 2-3 problems" — it's "be able to recognize this pattern instantly in a new, unseen problem." Revisit a topic's problems again a week later if the pattern didn't click the first time.

TIER 1 — MUST MASTER (High Interview Frequency)

These topics appear in the overwhelming majority of real coding interviews. Build deep fluency and pattern recognition here — this is non-negotiable for interview readiness.

0. Foundations Before Algorithms

- 0.1. What is an algorithm vs a data structure
- 0.2. Big-O, Big-Theta, Big-Omega notation — what they actually measure
- 0.3. How to derive time complexity by reading code (loops, nested loops, recursion)
- 0.4. Space complexity — what counts as extra space
- 0.5. Best case vs average case vs worst case
- 0.6. Amortized complexity (concept only, e.g. dynamic array resizing)
- 0.7. Recursion fundamentals — call stack, base case, recursive case
- 0.8. How to trace recursion by hand (recursion tree / call stack visualization)

PRACTICE — DO THESE BEFORE MOVING ON

Before touching real problems, practice ONLY complexity analysis: take 5 short code snippets (your own or from any source) and write down their Big-O by hand, then verify your reasoning by explaining it out loud. Also hand-trace a simple recursive function (e.g. factorial, Fibonacci) on paper, writing out the full call stack.

1. Arrays and Strings

- 1.1. Array fundamentals — indexing, traversal, in-place modification
- 1.2. Two pointers technique
- 1.3. Sliding window technique (fixed size and variable size)
- 1.4. Prefix sums

- 1.5. String manipulation basics (reversal, comparison, building with StringBuilder)
- 1.6. In-place array manipulation (rotation, partitioning)
- 1.7. Subarray / substring problems
- 1.8. Matrix/2D array traversal

PRACTICE — DO THESE BEFORE MOVING ON

HIGH IMPORTANCE — this is the single most-tested category in interviews. Practice: LeetCode #1 (Two Sum), LeetCode #167 (Two Sum II — sorted array, two pointers), LeetCode #3 (Longest Substring Without Repeating Characters, sliding window), LeetCode #238 (Product of Array Except Self, prefix/suffix), LeetCode #54 (Spiral Matrix, 2D traversal).

2. Hashing (Hash Maps and Hash Sets)

- 2.1. How a hash table works internally (concept-level: hashing, buckets, collisions)
- 2.2. Dictionary<TKey,TValue> and HashSet<T> in C# — when to use which
- 2.3. Using hash maps for frequency counting
- 2.4. Using hash maps for O(1) lookups to avoid nested loops
- 2.5. Hashing for detecting duplicates / grouping

PRACTICE — DO THESE BEFORE MOVING ON

HIGH IMPORTANCE — hashing is the most common technique for turning an $O(n^2)$ brute-force solution into $O(n)$. Practice: LeetCode #1 (Two Sum, revisit with hash map instead of two pointers), LeetCode #49 (Group Anagrams), LeetCode #242 (Valid Anagram), LeetCode #128 (Longest Consecutive Sequence).

3. Linked Lists

- 3.1. Singly linked list — structure, traversal, insertion, deletion
- 3.2. Doubly linked list
- 3.3. Fast and slow pointers (cycle detection, finding middle)
- 3.4. Reversing a linked list (iterative and recursive)
- 3.5. Merging linked lists

PRACTICE — DO THESE BEFORE MOVING ON

Practice: LeetCode #206 (Reverse Linked List), LeetCode #141 (Linked List Cycle, fast/slow pointers), LeetCode #21 (Merge Two Sorted Lists), LeetCode #19 (Remove Nth Node From End of List).

4. Stacks and Queues

- 4.1. Stack — LIFO concept, Stack<T> in C#, when to use
- 4.2. Queue — FIFO concept, Queue<T> in C#, when to use
- 4.3. Monotonic stack technique
- 4.4. Using a stack for matching/validation problems (parentheses, expressions)
- 4.5. Implementing a queue using two stacks (and vice versa) — concept

PRACTICE — DO THESE BEFORE MOVING ON

Practice: LeetCode #20 (Valid Parentheses), LeetCode #155 (Min Stack), LeetCode #739 (Daily Temperatures, monotonic stack), LeetCode #232 (Implement Queue using Stacks).

5. Recursion and Backtracking

- 5.1. Designing a recursive solution — identifying base case and recursive case
- 5.2. Backtracking — try, recurse, undo (the core pattern)
- 5.3. Generating combinations and permutations
- 5.4. Subsets / power set generation
- 5.5. Pruning — cutting branches early for efficiency

PRACTICE — DO THESE BEFORE MOVING ON

HIGH IMPORTANCE — backtracking underlies a large share of "medium/hard" interview problems. Practice: LeetCode #46 (Permutations), LeetCode #78 (Subsets), LeetCode #39 (Combination Sum), LeetCode #51 (N-Queens, harder — attempt after the others).

6. Binary Search

- 6.1. Classic binary search on a sorted array
- 6.2. Identifying when a problem is "binary searchable" even without an obviously sorted array
- 6.3. Binary search on answer (search space reduction pattern)
- 6.4. Common off-by-one mistakes and how to avoid them

PRACTICE — DO THESE BEFORE MOVING ON

HIGH IMPORTANCE — binary search appears far more often than its simplicity suggests, including in disguised forms. Practice: LeetCode #704 (Binary Search, classic), LeetCode #33 (Search in Rotated Sorted Array), LeetCode #153 (Find Minimum in Rotated Sorted Array), LeetCode #1011 (Capacity to Ship Packages, binary search on answer).

7. Sorting Algorithms

- 7.1. Built-in sorting in C# (Array.Sort, List.Sort, custom comparers)
- 7.2. Merge sort — mechanics and complexity
- 7.3. Quick sort — mechanics, complexity, and worst-case behavior
- 7.4. Why comparison-based sorting can't beat $O(n \log n)$ (concept only)
- 7.5. When sorting first simplifies an otherwise-hard problem

PRACTICE — DO THESE BEFORE MOVING ON

Practice: implement merge sort and quick sort from scratch by hand once each (this is one of the few cases where re-implementing a known algorithm matters for interviews). Then: LeetCode #56 (Merge Intervals, sort-first pattern), LeetCode #215 (Kth Largest Element in an Array).

8. Trees (Binary Trees and Binary Search Trees)

- 8.1. Tree terminology — root, leaf, height, depth, balanced vs unbalanced
- 8.2. Binary tree traversals — inorder, preorder, postorder (recursive and iterative)
- 8.3. Level-order traversal (BFS on a tree)
- 8.4. Binary Search Tree (BST) properties and operations (insert, search, delete)
- 8.5. Validating a BST
- 8.6. Lowest Common Ancestor (LCA) pattern
- 8.7. Tree height/balance/diameter problems

PRACTICE — DO THESE BEFORE MOVING ON

HIGH IMPORTANCE — trees are a top-3 most-tested category, especially in combination with recursion. Practice: LeetCode #104 (Maximum Depth of Binary Tree), LeetCode #98 (Validate Binary Search Tree), LeetCode #102 (Binary Tree Level Order Traversal), LeetCode #236 (Lowest Common Ancestor of a Binary Tree).

9. Heaps / Priority Queues

- 9.1. Heap property — min-heap vs max-heap
- 9.2. PriorityQueue<TElement,TPriority> in C#
- 9.3. Using a heap for "k largest/smallest" problems
- 9.4. Using two heaps for median-tracking problems (concept)

PRACTICE — DO THESE BEFORE MOVING ON

Practice: LeetCode #215 (Kth Largest Element in an Array, revisit with a heap approach), LeetCode #347 (Top K Frequent Elements), LeetCode #23 (Merge k Sorted Lists).

10. Graphs

- 10.1. Graph representations — adjacency list vs adjacency matrix
- 10.2. Directed vs undirected, weighted vs unweighted graphs
- 10.3. Breadth-First Search (BFS) — mechanics and use cases
- 10.4. Depth-First Search (DFS) — mechanics and use cases (recursive and iterative)
- 10.5. Detecting cycles in a graph
- 10.6. Connected components
- 10.7. Topological sort (concept and use cases)

PRACTICE — DO THESE BEFORE MOVING ON

HIGH IMPORTANCE — graphs are common in mediums/hards and are frequently disguised as grid problems. Practice: LeetCode #200 (Number of Islands, grid BFS/DFS), LeetCode #133 (Clone Graph), LeetCode #207 (Course Schedule, topological sort / cycle detection), LeetCode #133 or #547 (Number of Provinces, connected components).

11. Dynamic Programming

- 11.1. Identifying when a problem has the DP property (overlapping subproblems, optimal substructure)
- 11.2. Memoization (top-down DP)
- 11.3. Tabulation (bottom-up DP)
- 11.4. 1D DP patterns (e.g. climbing stairs, house robber)
- 11.5. 2D DP patterns (e.g. grid paths, longest common subsequence)
- 11.6. Knapsack pattern (0/1 knapsack and variations)
- 11.7. Converting a brute-force recursive solution into a DP solution step by step

PRACTICE — DO THESE BEFORE MOVING ON

HIGH IMPORTANCE — DP is consistently rated the hardest category by interview candidates and is heavily weighted in "hard" interview rounds. Practice: LeetCode #70 (Climbing Stairs), LeetCode #198 (House Robber), LeetCode #62 (Unique Paths, 2D DP), LeetCode #322 (Coin Change, unbounded knapsack-style), LeetCode #1143 (Longest Common Subsequence).

12. Greedy Algorithms

- 12.1. The greedy choice property — what makes greedy valid for a problem
- 12.2. Recognizing when greedy works vs when it gives a wrong answer (vs needing DP)
- 12.3. Interval scheduling problems
- 12.4. Common greedy patterns (e.g. always pick smallest/largest available option)

PRACTICE — DO THESE BEFORE MOVING ON

Practice: LeetCode #455 (Assign Cookies), LeetCode #435 (Non-overlapping Intervals), LeetCode #122 (Best Time to Buy and Sell Stock II).

TIER 2 — SHOULD KNOW (Occasional Interview Frequency)

These show up periodically, often in specific company patterns (e.g. companies with heavy systems/infra focus) or as the harder problem in a round. Understand them solidly, but they don't need the same repetition volume as Tier 1.

- Tries (prefix trees) — used for word search, autocomplete-style problems (LeetCode #208, Implement Trie)
- Union-Find / Disjoint Set Union (DSU) — used for connectivity/grouping problems, an alternative to graph traversal for some problems (LeetCode #547, Number of Provinces)
- Advanced graph algorithms — Dijkstra's algorithm (shortest path, weighted graph), Union-Find with weighted edges, Bellman-Ford (concept only)
- Bit manipulation — AND/OR/XOR tricks, bit masking, counting set bits (LeetCode #136, Single Number)
- Interval merging/scheduling beyond the basics — meeting rooms style problems
- Sliding window maximum (deque-based technique) — a specific advanced variant of sliding window
- Segment trees and Fenwick trees (concept-level awareness; full implementation is rare in standard interviews)
- String algorithms beyond basics — KMP pattern matching, Rabin-Karp (concept-level; rarely required to implement from scratch)
- Multi-source BFS (e.g. rotting oranges-style problems)
- DP on trees / DP on graphs (a more advanced fusion of two Tier 1 topics)
- Math-heavy problems — GCD/LCM, prime sieve (Sieve of Eratosthenes), modular arithmetic

TIER 3 — AWARE OF ONLY (Rare in Standard Interviews)

These are classic CS curriculum topics that rarely appear directly in standard coding interviews (outside of specialized roles like systems/database engineering). Know the name and rough idea; don't dedicate study time unless a specific company/role signals otherwise.

Red-Black Trees / AVL Trees (full implementation)

Self-balancing BST variants. You should know that BSTs need self-balancing to guarantee $O(\log n)$, and that these are the classic ways to do it — but implementing one from scratch in an interview is extremely rare.

B-Trees

Used internally in databases and filesystems for disk-based storage. Relevant to know exists if asked about how databases index data, not for implementation.

Skip Lists

A probabilistic alternative to balanced trees, used in some real systems (e.g. Redis). Rarely tested directly.

Advanced string algorithms (Z-algorithm, Suffix Arrays/Trees)

Used in specialized text-processing/bioinformatics contexts. Essentially never required in standard interviews.

Network flow algorithms (Max Flow / Min Cut)

Graph theory topics relevant to specific competitive programming or specialized optimization roles, not typical interviews.

NP-Completeness / Computational Complexity Theory

Useful to recognize a problem is NP-hard (so you don't waste time hunting for a polynomial solution that doesn't exist), but proving reductions is academic-only territory for interview purposes.

Treaps, Splay Trees, and other exotic tree structures

Exist in specialized contexts; recognize the names if mentioned, no need to study.

Note on ordering: Tier 1 sections are listed in a dependency-aware order — recursion (Section 5) is placed before trees/graphs/DP because those topics lean on recursive thinking heavily, and binary search (Section 6) is placed before DP since DP problems often build on search-space reasoning. Within each topic, do the listed practice problems on the named platform before moving on — reading alone will not build interview-speed pattern recognition. If a problem from Tier 1 takes more than ~40 minutes with no progress, check a hint rather than the full solution, and revisit the problem cold a few days later.